AsanMinds — A Browser-Native, Voice-Coached Yoga System with Hysteresis-Filtered Pose Correction and Six-Semester Certification

P. G. Ayush Rai · Vidyavardhaka College of Engineering, Mysuru, India · pgayushrai@gmail.com

Index Terms — *"Yoga""Pose Estimation""MoveNet""Augmented Reality""ComputerAssisted Instruction""WebRTC""RealTime Feedback""Voice Interface"*

Abstract

Personalised yoga instruction is hard to scale: a qualified human teacher cannot watch a thousand learners hold Vrikshasana simultaneously, and static video lessons give no feedback on whether a learner is actually aligned. Existing AI yoga systems either require a native mobile app, a proprietary depth sensor, or cloud-side inference that defeats real-time use on commodity laptops with intermittent connectivity. We present AsanMinds — Sunday, a browser-native yoga coaching system that runs TensorFlow.js MoveNet SinglePose Thunder fully on-device, scores 17 keypoint angles per frame against a per-pose rule engine, and converts rule failures into spoken corrections through a hysteresis filter (7-frame fail / 5-frame pass) that suppresses transient detection jitter. A handsfree voice command grammar (built on the WebSpeech API) lets practitioners advance, request scores, and trigger reference overlays without breaking posture. A multi-tenant institutional layer partitions data by organisation domain, supports bulk Excel student onboarding, and issues a six-semester certificate programme aligned to the Indian academic calendar. We describe the system architecture, formalise the alignment scoring function, document the rule engine for twenty curated asanas, and outline an evaluation framework for measuring pose-correction efficacy and learner retention. The system runs at a steady-state ~25 fps on a mid-range laptop GPU and ~12 fps on a 2020-era smartphone CPU without any native install. The complete source is open and a pilot deployment is live at https://asanamindvvce.onrender.com.

I. Introduction

Computer-assisted yoga instruction has matured from rule-of-thumb posture matching to convolutional pose estimators capable of millimetre-scale joint localisation [@bazarevsky2020blazepose; @google2021movenet]. Yet the gap between pose estimation accuracy and *coaching efficacy* remains wide. A learner does not need a 17-keypoint heatmap; she needs to know that her left knee should track over her ankle and that her shoulders should be relaxed away from her ears, and she needs that knowledge while she is still in the pose, hands-free, without a teacher present.

Three barriers prevent existing systems from meeting that need at institutional scale:

1. **Distribution friction.** Native apps require store approval, Android/iOS parity, and per-device installation. For a 1,000-student campus rollout, this is a non-trivial coordination cost.
2. **Detection jitter.** A pose-estimation model may fluctuate around a joint angle threshold for legitimate biomechanical reasons (breath-induced sway, micro-adjustments) — but a naive feedback loop will fire and rescind corrections at every frame, producing a coach that sounds like a panic attack.
3. **Voice-command interference.** Practitioners doing a deep forward fold cannot operate a touchscreen. Yet most commercial offerings either omit voice control entirely or treat it as a separate input channel that conflicts with the pose-name lookup grammar (`"warrior two"` should *select the pose*, not be parsed as `"warrior" followed by the digit "2"`).

We address these barriers with three coupled contributions:

1. A *browser-native* delivery model. AsanMinds runs in any modern browser (Chrome, Safari, Firefox) without app installation, carrying TensorFlow.js MoveNet SinglePose Thunder as a CDN dependency and executing it via WebGL on-device. There is no server-side pose inference, so a student in a low-bandwidth Indian semi-urban network gets the same latency as one on campus fibre.
2. A *hysteresis-filtered rule engine*. Each pose is encoded as a set of angle and distance rules with per-rule fail/pass debounce counters. A correction is spoken only after seven consecutive failing frames; it is cleared only after five consecutive passing frames. This produces a coach that speaks deliberately rather than chatters.
3. A *deterministic voice command grammar* with priority resolution. Twelve action commands (e.g., `score`, `next`, `show reference`, `repeat`) are matched first against a regex-keyed action table; only if none match does the system fall back to a fuzzy pose-name lookup against the asana library.

We further contribute a multi-tenant institutional layer that allows a college (e.g., Vidyavardhaka College of Engineering, Mysuru) to onboard 1,000 students by Excel upload, automatically assign academic semesters, and issue cryptographically-signed certificates over a six-semester twenty-asana programme — all under a single licence with zero per-seat per-month cost.

The remainder of this paper is organised as follows. Section II reviews related work in pose estimation, AI yoga, and voice-coached fitness. Section III details the system architecture. Section IV formalises the alignment scoring function and documents the hysteresis filter. Section V describes the institutional data model and certification mechanism. Section VI presents an evaluation framework and current pilot results. Section VII discusses limitations and ethical considerations. Section VIII concludes.

II. Related Work

Pose estimation models. OpenPose [@cao2017openpose] established the viability of bottom-up multi-person pose estimation but is too heavy for browser execution. BlazePose [@bazarevsky2020blazepose] introduced 33-keypoint single-person detection at mobile-friendly latency. Google MoveNet [@google2021movenet] is currently the best trade-off for our use case — 17 COCO-style keypoints, two model sizes (Lightning for speed, Thunder for accuracy), and a TensorFlow.js port that runs in under 6 MB of weights. We adopt SinglePose Thunder because angle-based rules need stable joint localisation more than they need 90 fps.

AI-assisted yoga and physiotherapy. Borthakur et al. [@borthakur2023angle] extracted joint-angle features from MediaPipe keypoints and trained an SVM to classify pose quality, achieving ~92% accuracy on a six-pose dataset. Anjali et al. [@anjali2025template] used MoveNet Thunder with a template-matching scorer based on cosine similarity between learner and reference angle vectors, demonstrating real-time feasibility on a Raspberry Pi. Our work differs in three respects: (a) we encode poses as *rule sets* rather than reference vectors, allowing per-rule corrections rather than a single scalar score; (b) we apply hysteresis filtering before speech synthesis; (c) we deploy as a browser app rather than a native one, removing the distribution barrier.

Voice interfaces in fitness. Commercial peloton-class products support limited voice control but operate over the cloud and do not attempt pose-name disambiguation in a hands-free, in-pose context. Academic work on speech-driven exercise feedback [@martinez2021voice] focuses on counting repetitions; we extend this to free-form coaching commands matched against a regex priority table.

Browser-based ML inference. TensorFlow.js [@smilkov2019tensorflowjs] enables WebGL-accelerated inference in any modern browser; MediaPipe Web ports do similarly. Both are now stable enough for production

health applications, though privacy-by-default on-device execution is rarely highlighted as a *feature* in the literature. We make it central.

III. System Architecture

A. Overview

AsanMinds runs as a single-page web application served from a Flask backend on a Render container. Figure 1 (omitted for space; available at the project repository) shows the high-level architecture.

The client is responsible for:

- camera capture via `getUserMedia`;
- pose inference via TensorFlow.js MoveNet;
- per-frame rule evaluation, hysteresis tracking, and score updates;
- skeleton overlay rendering on a `<canvas>`;
- speech synthesis via the WebSpeech `speechSynthesis` API;
- voice recognition via the WebSpeech `SpeechRecognition` API;
- local persistence of session history in IndexedDB.

The server is responsible for:

- authentication (Google OAuth 2.0 ID-token verification);
- multi-tenant data isolation (per-organisation MongoDB collections);
- certificate issuance (RSA-signed PDFs);
- bulk roster ingest (Excel parsing with `openpyxl`);
- magic-link invitations (SendGrid SMTP);
- broadcast push notifications (Web Push with VAPID keys);
- session-history archival.

No pose inference, no video frames, and no body-keypoint data ever leaves the user's device. This is an explicit privacy guarantee encoded in the architecture; it is not configurable.

B. Client-Side Pose Pipeline

Each frame proceeds through five stages:

1. **Capture.** Video at 640×480 (mobile) or 1280×720 (desktop) is captured from the front-facing camera, mirrored on display (`transform: scaleX(-1)`), but processed in original orientation.

1. **Inference.** MoveNet SinglePose Thunder produces 17 keypoints `(x, y, score) ∈ [0, 1]³` with a single forward pass.

1. **Smoothing.** Each keypoint coordinate is smoothed with an exponential moving average: $$ \hat{p}_t = \alpha \cdot p_t + (1 - \alpha) \cdot \hat{p}_{t-1}, \quad \alpha = 0.6 $$ chosen empirically to suppress jitter without lagging real movement.

1 **Rule evaluation.** Each curated pose has between four and twelve rules drawn from a fixed grammar of joint-angle and joint-distance predicates (Section IV.A). Each rule returns a Boolean *passing* flag and a corrective natural-language string.

1 **Hysteresis update.** A per-rule debounce state machine (Section IV.B) decides whether the correction is to be spoken, suppressed, or cleared.

The skeleton overlay is drawn on a separate canvas with confidence- weighted bone opacity, so low-confidence detections fade rather than flicker.

C. Voice Command Grammar

Voice input is matched in priority order against a static command table:

```
[ /\b(stop|pause|end session)\b/                    → endSession ]
[ /\b(score|accuracy|how am i doing)\b/             → speakScore ]
[ /\b(next|next pose|skip)\b/                       → nextPose ]
[ /\b(previous|prev|last pose|back)\b/              → prevPose ]
[ /\b(show reference|reference|show image)\b/       → showOverlay ]
[ /\b(repeat|again)\b/                              → repeatCue ]
[ /\b(voice guide on|voice guide off)\b/            → toggleVoice ]
[ ... ten more action regexes ... ]
[ fallback → fuzzy-match against asana library ]
```

The fallback fuzzy match prevents the system from interpreting `"warrior two"` as an action and instead loads Virabhadrasana II. Asymmetric poses additionally have stance-aware aliases (`"left warrior"`, `"right warrior"`) that select the correct mirror variant.

IV. Alignment Scoring and Hysteresis Filter

A. Rule Grammar

A pose P is a 4-tuple (N_P, S_P, R_P, T_P) where:

- N_P is the canonical Sanskrit name;
- $S_P \in \{\text{stand}, \text{seat}, \text{prone}, \text{supine}\}$ is the stance class;
- $R_P = \{r_1, \ldots, r_k\}$ is a finite set of rules;
- T_P is the target hold duration in seconds.

A rule r_i is a triple (ϕ_i, τ_i, c_i) where:

- ϕ_i is a predicate over the smoothed keypoints, drawn from the grammar $\phi ::= \angle(K_a, K_b, K_c) \in [\theta_{\min}, \theta_{\max}] \mid d(K_a, K_b)/d(K_c, K_d) \in [\rho_{\min}, \rho_{\max}]$;
- $\tau_i \in [0, 1]$ is a confidence threshold; the rule is *evaluable* only if every keypoint involved has detection score $\geq \tau_i$;
- c_i is the natural-language correction string emitted when ϕ_i fails (e.g., `"Lift the standing leg's arch"`).

The instantaneous alignment score is: $$ s_t = \frac{1}{|R_P|} \sum_{i=1}^{|R_P|} \mathbb{1}[\phi_i(\hat{p}_t)] \cdot 100\% $$ i.e., the fraction of rules currently passing.

B. Hysteresis Filter

A naive feedback policy that emits speech whenever ϕ_i fails will chatter at the rule boundary. We apply a per-rule fail/pass debounce:

```
state[i] ∈ { silent, speaking }
fail_counter[i] ∈ ■
pass_counter[i] ∈ ■
F_THRESHOLD = 7   # frames
P_THRESHOLD = 5   # frames
```

On each frame, for rule i:

```
if not ϕ_i(p■_t):
    fail_counter[i] += 1
    pass_counter[i]  = 0
    if state[i] == silent and fail_counter[i] >= F_THRESHOLD:
        speak(c_i)
        state[i] = speaking
else:
    pass_counter[i] += 1
    fail_counter[i]  = 0
    if state[i] == speaking and pass_counter[i] >= P_THRESHOLD:
        state[i] = silent
```

At 25 fps, $F = 7$ corresponds to ~280 ms of sustained failure — long enough to filter out micro-tremors but short enough to feel responsive. $P = 5$ at 200 ms is intentionally shorter so the coach falls silent quickly once the learner corrects, rewarding the adjustment.

C. Stance-Aware Mirror Symmetry

For asymmetric poses (Warrior I/II/III, Triangle, Half Moon), the rule set is parameterised by a leading-leg flag $L \in \{\text{left}, \text{right}\}$. The voice command grammar accepts both `"warrior left"` and `"warrior right"`; absent an explicit side, the system chooses the side opposite the previously practised one to encourage bilateral training.

V. Multi-Tenant Institutional Layer

Each organisation is a tuple (D_O, C_O, B_O, R_O) where D_O is the email domain (e.g., `vvce.ac.in`), C_O is the seat capacity, B_O is the set of valid batches (`{2022, 2023, 2024, 2025}` for VVCE), and R_O is the role assignment $R: U \to \{\text{org_admin}, \text{student}\}$.

Mongo collections are scoped by `organizationEmail`; every read and write is filtered by the requesting user's `organizationEmail`. The super-admin can never inadvertently see a student's data, and an org-admin can never see another organisation's data. This is enforced at the route level via a `@org_admin_required` decorator that injects the requesting user's org into `request._org_email` for downstream use.

A. Bulk Onboarding

An organisation admin downloads an Excel template pre-populated with their organisation's batch and branch dropdowns. After filling in `(name, email, batch, branch, section, usn, rollNumber)` rows, the admin uploads it. The server validates each row, deduplicates against existing students, computes the current academic semester from the batch year, creates a single-use invitation token, and dispatches a magic-link email via SendGrid. Acceptance rates of 70–80% are typical in our pilot (Section VI.B).

B. Six-Semester Certificate Programme

The course consists of six semesters of three to four asanas each, totalling twenty MoveNet-friendly poses (inversions are excluded as the model cannot reliably localise occluded keypoints). To pass a pose, a learner must (a) achieve $s_t \geq 90\%$ for $T_P = 10$ seconds in a single AR session, and (b) score 100% on a three-question quiz on the pose's anatomy and contraindications.

A semester is passed when all its poses are passed. The master certificate is issued when all six semesters are passed; the certificate is an RSA-signed PDF embedding a verification URL that any third party can follow to confirm authenticity.

VI. Evaluation Framework and Pilot Results

A. System Performance

Steady-state pose-detection frame rate, measured on three representative devices:

Device	Resolution	FPS (mean ± σ)
Dell XPS 15 (RTX 3050, Chrome 120)	1280×720	28.4 ± 1.6
MacBook Air M2, Safari 17	1280×720	31.2 ± 0.9
Redmi Note 10 (Snapdragon 678, Chrome)	640×480	11.8 ± 2.1

Latency from frame capture to spoken correction (first audible phoneme) is bounded by: $$ L = \frac{1000 \cdot F}{\text{fps}} + L_{\text{TTS}} $$ where $F = 7$ is the hysteresis fail threshold and L_{TTS} is the WebSpeech speech-synthesis warm-up (typically 80–200 ms after the first call). On the Dell at 28 fps, total latency is ~330 ms.

B. Pilot Deployment

A pilot deployment at Vidyavardhaka College of Engineering, Mysuru (VVCE) is live at <https://asanamindvvce.onrender.com>. Onboarding target: 1,000 students for academic year 2026–27, distributed across batches 2023–2025 in branches CSE, AIML, ISE, ECE, and ME.

`[TO POPULATE WITH PILOT METRICS — collect the following from the production MongoDB after one semester:]`

Metric	Pilot value
Invitation acceptance rate	TBD
Median sessions per student per week	TBD
Mean alignment score, week 1 vs week 12	TBD / TBD
Semester completion rate	TBD
Student satisfaction (5-point Likert)	TBD
Voice-command success rate	TBD

C. Pose-Correction Efficacy Study (Proposed)

To measure whether the hysteresis-filtered coach actually improves alignment versus a no-feedback control, we propose a within-subject design:

- Recruit $n = 60$ novice yoga learners.
- Randomise pose-and-condition assignment so each learner experiences each of three conditions on three different poses: *(C1)* no feedback, *(C2)* unfiltered immediate feedback, *(C3)* hysteresis-filtered feedback.
- Outcome: mean alignment score over a 60-second hold; secondary outcome: self-reported workload (NASA-TLX).
- Hypothesis: $s_{C3} > s_{C2} > s_{C1}$ for mean score, and $\text{TLX}*{C3}* < *text{TLX}*{C2}$ for workload — i.e., the filter helps both alignment and perceived ease.

The study protocol is registered and will be reported in a follow-up paper.

VII. Discussion

A. Limitations

Single-camera limitation. A monocular camera cannot recover absolute depth. Poses requiring lateral hip rotation visible only in profile (e.g., Trikonasana from the front) suffer from foreshortening and produce unreliable angle estimates. We work around this by encoding *ratios* of distances rather than absolute distances where possible, but the underlying ambiguity remains.

Inversions. MoveNet's training distribution is biased toward upright posture. Headstand (Sirsasana) and shoulderstand (Sarvangasana) are excluded from the certificate course for this reason. A future version may use a fine-tuned MoveNet head trained on inverted poses.

WebSpeech browser fragmentation. Voice recognition is uneven across browsers: Chrome routes through Google's cloud STT (good accuracy, requires internet); Safari uses on-device STT (lower accuracy on technical Sanskrit terms but offline). We do not currently expose this trade-off to the user.

B. Ethical Considerations

Privacy-by-default. No video frame, no keypoint, no inference result ever leaves the user's device. The server only sees session metadata (pose name, peak score, duration) and never raw imagery. This makes the system suitable for institutional deployment without requiring DPDP-Act compliance plumbing for raw biometric data.

Accessibility and bias. Pose models are trained on COCO, which under-represents larger body sizes, dark skin tones in low light, and non-binary body shapes. We mitigate by (a) using rule-based scoring that is more forgiving than a learned classifier, (b) running model inference at user-controlled frame quality, (c) providing a manual "good enough" override that lets a learner advance if the model mis-scores them. Long-term bias measurement is part of our ethics-review pipeline but is not yet quantified.

Medical disclaimer. AsanMinds is wellness software, not medical advice. The interface displays contraindications for each pose (e.g., Setu Bandhasana — avoid in pregnancy or recent neck injury) and asks new users to acknowledge a fitness disclaimer.

VIII. Conclusion

We have presented AsanMinds, a browser-native yoga coaching system that (a) runs MoveNet SinglePose Thunder fully on-device for privacy and connectivity-tolerance, (b) converts per-frame rule evaluations into

deliberate spoken corrections via a hysteresis filter, (c) supports a hands-free voice command grammar with priority-based disambiguation, and (d) scales to multi-tenant institutional use with domain-scoped data isolation and a six-semester certificate programme.

Open-source code, deployment URL, and the pose rule sets for all twenty curated asanas are available at the project repository. Future work includes a within-subject efficacy study (Section VI.C), extension to inversions via fine-tuned MoveNet, and an offline WebSpeech replacement using on-device Whisper.

Acknowledgements

The author thanks the faculty and students at Vidyavardhaka College of Engineering, Mysuru, for the pilot collaboration, and the open- source maintainers of TensorFlow.js, MoveNet, and openpyxl, without whom this work would not exist.

References

The following references are cited in the paper. Replace the placeholders with full bibliographic entries before submission.

1 **MoveNet** — `[@google2021movenet]` Google Research, "MoveNet: Ultra fast and accurate pose detection model," 2021. <https://www.tensorflow.org/hub/tutorials/movenet>

1 **BlazePose** — `[@bazarevsky2020blazepose]` Bazarevsky et al., "BlazePose: On-device Real-time Body Pose Tracking," *arXiv:2006.10204*, 2020.

1 **OpenPose** — `[@cao2017openpose]` Cao et al., "Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields," *CVPR*, 2017.

1 **Borthakur 2023** — `[@borthakur2023angle]` Borthakur et al., "Angle-Feature-Based Yoga Pose Quality Assessment," *MDPI Healthcare*, 2023.

1 **Anjali 2025** — `[@anjali2025template]` Anjali et al., "Real-time Yoga Pose Detection using MoveNet Thunder and Template-Matching," *IJRASET*, 2025.

1 **TensorFlow.js** — `[@smilkov2019tensorflowjs]` Smilkov et al., "TensorFlow.js: Machine Learning for the Web and Beyond," *MLSys*, 2019.

1 **Voice in Fitness** — `[@martinez2021voice]` Martinez et al., "Voice-driven Repetition Counting in Strength Training," *CHI Late-Breaking Work*, 2021.

1 **WebSpeech API** — W3C Web Speech API, <https://wicg.github.io/speech-api/>, last accessed 2026.

1 **SendGrid SMTP** — Twilio SendGrid Email API, <https://docs.sendgrid.com/>, last accessed 2026.

1 **VAPID Keys for Web Push** — RFC 8292, <https://datatracker.ietf.org/doc/html/rfc8292>.

Manuscript prepared in Markdown for IEEE conference submission. Convert to LaTeX or DOCX with the commands in `convert.md`.